
FinWise AI Project Report

Abstract

FinWise AI is a local personal finance management system that helps a user track bank accounts, cash, UPI wallets, credit cards, loans, income, expenses, transfers, budgets, savings goals, analytics, reports, statement imports, and finance guidance. The project is designed as an offline-first application, so the main finance features and the built-in AI assistant can work without an internet connection after the required software is installed.

The system uses a React frontend, Flask backend, MySQL database, and explainable finance algorithms. The AI part is not a cloud chatbot. It uses local rules, user transaction history, and a Multinomial Naive Bayes category prediction model to answer finance-related questions and help classify transactions.

Problem Statement

Many students and individuals record money details manually or keep them spread across bank apps, notes, and spreadsheets. This makes it difficult to know:

- How much money is currently available.
- Where money is being spent.
- How much was earned and spent in a selected month.
- Whether a budget is being crossed.
- How much has been saved toward goals.
- How loans, credit cards, and transfers affect account balances.

FinWise AI solves this by providing one simple finance dashboard with transaction tracking, account-wise ledger, analytics, statement import, and offline finance guidance.

Objectives

- Track total money, balance money, income, expenses, savings, transfers, loans, and credit cards.
- Maintain multiple account types such as bank, cash, UPI wallet, credit card, loan, and investment.
- Import bank statements from CSV, Excel, and text-based PDF files.
- Detect duplicate imported transactions before saving.
- Provide month-wise analytics and category-wise expense breakdown.
- Maintain a ledger with debit, credit, and running balance.
- Support budgets and savings goals.
- Provide an offline finance chat assistant for project and finance questions.
- Use simple, explainable algorithms suitable for an academic project.

Scope

The project focuses on personal finance tracking for one user account at a time. It supports local database storage and local financial analysis. It does not connect directly to real bank APIs, does not send real emails without provider setup, and does not require a paid online AI service.

Technology Stack

Frontend

- React for the user interface.
- Vite for frontend build.
- Bootstrap and Bootstrap Icons for layout and icons.
- Chart.js for charts and analytics.

Backend

- Flask for REST API services.
- MySQL Connector for database communication.
- JWT and httpOnly cookies for login sessions.
- Werkzeug password hashing for secure password storage.
- openpyxl and xlrd for Excel statement files.
- pdfplumber for text-based PDF bank statements.
- reportlab for report generation support.

Database

- MySQL stores users, accounts, transactions, budgets, savings goals, goal contributions, statement import history, preferences, reports, and automation records.

Main Modules

1. Authentication and Profile

The system supports registration, login, logout, profile viewing, profile editing, password change, and theme preferences. Passwords are hashed before being stored. Sessions use httpOnly cookies, which is safer than storing login tokens directly in browser localStorage.

2. Dashboard

The dashboard shows the most important finance summary:

- Total Money: balance money plus saved goal money.
- Balance Money: available amount in accounts.
- Income: money received.
- Expense: money spent.
- Savings: money saved in goals.

It also shows monthly overview, recent transactions, budget risk, and quick actions.

3. Accounts

The accounts module supports different account types:

- Bank
- Cash

-
- UPI Wallet
 - Credit Card
 - Loan
 - Investment

This is useful because a user can track savings accounts, loan balances, and credit card balances in the same project while still keeping them separated by account type.

4. Transactions

Transactions can be added manually as:

- Income
- Expense
- Transfer

Income increases an account balance. Expense reduces an account balance. Transfer moves money from one account to another without counting it as income or expense.

5. Bank Statement Import

The statement import module allows the user to upload CSV, Excel, or text-based PDF statement files. The system previews rows before importing. The user can correct date, description, type, category, and amount before saving.

The import feature also checks possible duplicate rows so the same bank statement does not create repeated transactions.

6. Ledger

The ledger module gives an account-wise debit and credit view. It calculates a running balance for each account.

Ledger rule:

$$\text{Running Balance} = \text{Opening Balance} + \text{Credits} - \text{Debits}$$

For a transfer, the ledger creates two movements:

- Debit from the source account.
- Credit to the destination account.

This makes the project stronger because it uses accounting-style transaction tracking.

7. Budgets

The budget module lets the user set category-wise spending limits. The system compares actual expense against the budget limit and can show when spending is close to or above the limit.

8. Savings Goals

Savings goals help the user move money from an account into a goal. This is not treated as a normal expense because the money is still owned by the user. It is tracked separately as savings.

Example:

- Moving Rs 1000 to a phone savings goal reduces available account balance.
- It increases saved goal money.
- It does not count as food, bills, rent, or other expense.

9. Analytics

Analytics gives selected-month checking. The user can compare income, expense, net flow, saved-to-goal money, top category, category chart, and transactions for a selected month.

10. Reports

Reports help export or view summarized financial data. This is useful for checking monthly progress and explaining finance history.

11. Smart Tools

Smart Tools includes auto entries for repeated income or expenses such as salary, rent, subscriptions, EMI, and bills. This avoids entering the same transaction manually every month.

12. Offline Finance Chat

The chat assistant answers finance and project-related questions locally. It can explain:

- Total money, balance money, income, expense, and savings.
- Transfers between accounts.
- Ledger and running balance.
- Algorithms used in the project.
- How to use accounts, loans, credit cards, budgets, and goals.
- Whether the app works offline.

The chat does not require internet or an online AI API.

Algorithms Used

1. Multinomial Naive Bayes Category Prediction

This algorithm predicts a transaction category from the transaction description. It uses finance keywords and the user's previously saved transactions.

Example:

```
"swiggy dinner" -> Food
"salary july" -> Salary
"electricity bill" -> Bills
```

Basic idea:

Predicted Category = category with highest $P(\text{category} \mid \text{words})$

The algorithm calculates:

$P(\text{category} \mid \text{words})$ proportional to $P(\text{category}) * P(\text{word1} \mid \text{category}) * P(\text{word2} \mid \text{category}) \dots$

This is explainable and suitable for a simple AI-based academic project.

2. Rule-Based Offline Chat Intent Matching

The finance chat checks the user's question for important keywords and matches it to local help topics and finance rules.

Examples:

- "is this offline" -> explains offline working.
- "what algorithm used" -> explains Naive Bayes, ledger, duplicate detection, and analytics formulas.

-
- "transfer between accounts" -> explains Transfer transaction type.
 - "what is balance money" -> explains available account balance.

This is simple, fast, offline, and explainable.

3. Ledger Running Balance Algorithm

The ledger algorithm orders account movements by date and calculates the balance after every transaction.

```
opening = account opening balance
for each ledger row:
    running_balance = running_balance + credit - debit
```

This makes it easy to verify how each transaction changed the account balance.

4. Duplicate Transaction Detection

During statement import, the system checks whether a similar transaction already exists. It compares important fields such as account, date, amount, description, and transaction type. Duplicate rows are marked before import so the user does not accidentally import the same statement again.

5. Budget Utilization Algorithm

The system compares expense against budget:

```
Budget Usage Percent = (Actual Expense / Budget Limit) * 100
```

This helps identify categories where the user is overspending.

6. Savings Rate Calculation

The system calculates savings performance using income and savings/remaining money.

```
Savings Rate = ((Income - Expense) / Income) * 100
```

This helps the user understand whether spending is controlled.

7. Goal Progress Algorithm

Each savings goal has a target and saved amount.

```
Goal Progress Percent = (Saved Amount / Target Amount) * 100
```

This tells how close the user is to completing a financial goal.

8. Month-Wise Analytics Algorithm

The analytics page filters transactions by selected month and groups them by type and category.

It calculates:

- Monthly income.
- Monthly expense.
- Net flow.
- Top expense category.
- Goal savings movement.
- Category chart values.

9. Financial Health Rules

The project can judge finance condition using simple rules such as:

-
- Spending is high if expenses are close to or above income.
 - Budget risk is high if category spending crosses the limit.
 - Savings progress is good if goal savings are increasing.
 - Available balance should be checked separately from total tracked money.

These rules make the project explainable to a guide.

Database Overview

Important database tables include:

- users: stores registered user details and hashed passwords.
- accounts: stores bank, cash, UPI, loan, credit card, and investment accounts.
- transactions: stores income, expense, and transfer records.
- budgets: stores category-wise monthly limits.
- savings_goals: stores goals and targets.
- goal_contributions: stores money moved into or out of goals.
- statement_imports: stores uploaded statement import history.
- recurring_transactions: stores auto-entry rules.
- user_preferences: stores theme and display preferences.
- password_reset_tokens: stores hashed reset tokens.
- email_verification_tokens: stores hashed email verification tokens.

Some future-ready tables may exist for advanced features, but the main user interface focuses on simple personal finance tracking.

Data Flow

```
User action in React UI
    |
    v
Flask API receives request
    |
    v
Authentication checks current user
    |
    v
Business logic applies finance rules
    |
    v
MySQL stores or reads data
    |
    v
React UI displays dashboard, tables, charts, and chat answer
```

Security Features

- Passwords are hashed using Werkzeug.
- Login uses httpOnly cookies.
- Reset and verification tokens are hidden by default.

-
- Local secrets should be stored in ``.env``.
 - User data is filtered by logged-in user ID.
 - Statement import preview lets the user review data before saving.

Offline Working

FinWise AI can run locally on the same computer using:

- Local React build.
- Local Flask server.
- Local MySQL database.
- Local rule-based finance chat.
- Local Naive Bayes category prediction.

Internet is only needed for first-time installation or if new packages must be downloaded. After setup, the project can run without internet.

Limitations

- It does not directly connect to real bank APIs.
- PDF import works best for text-based PDFs, not scanned image PDFs.
- Email invite and email reset delivery need a real email provider if used.
- The AI assistant is explainable and offline, but it is not a large online AI model.
- Mobile app support is not included.

Future Enhancements

- Bank API integration for automatic bank sync.
- OCR support for scanned PDF statements.
- Mobile application version.
- Advanced forecasting using linear regression or time series models.
- More detailed loan EMI and credit card interest calculations.
- Optional cloud AI integration for deeper natural language answers.

Testing and Validation

The project includes safety tests for important behavior:

- Weak default secrets are avoided.
- Reset tokens are hidden by default.
- Browser popup calls are avoided in frontend pages.
- Statement undo does not bulk delete older transactions.
- Frontend dependencies are pinned.
- Offline chat includes project help topics.

-
- Family option is removed from frontend.
 - Ledger feature is connected to backend and frontend.

The frontend build also validates that the React interface compiles successfully.

How to Run

Open the project folder:

```
cd C:\FinWise_AI
```

Start the application:

```
.\run.cmd
```

Open:

```
http://127.0.0.1:5000
```

Keep the run window open while using the app.

Conclusion

FinWise AI is a simple but complete personal finance management system. It tracks real financial activities such as income, expenses, transfers, loans, credit cards, budgets, goals, and account ledger. The project includes offline AI features using explainable algorithms, which makes it suitable for academic demonstration. The system is practical, understandable, and can be extended with more advanced financial algorithms in the future.